

Funzioni Higher-Order su liste

- abbiamo visto che la ricorsione è l'unico modo con cui possiamo ripetere un calcolo + volte

High-order: funzioni che prendono come parametro o restituiscono una funzione

- per scombinare o elaborare le liste è inevitabile ricorrere alla ricorsione es MAP() in javascript

Esempi

```
in []: let rec contiene_zero lis =  
  match lis with  
  | [] -> False  
  | x::lis' -> if x=0 then true  
                else contiene_zero lis;;  
|  
|  
|
```

- l'unica differenza tra le funzioni è la guardia dell'if
questo serve il parametro una funzione "predicato" cioè funzione che restituisce true o false

Exist

```
let rec exist p lis =  
  match lis with  
  | [] -> false  
  | x::lis' -> if p x then true  
                else exists p lis;;
```

ORA POSSIAMO RIDEFINIRE CONTIENE ZERO

let contieneZero lis = exists (fun x → x = 0) lis

// exist qualunque condizione gli diamo ora potremo fare la ricorsione della lista

contains // verifica che un certo elemento è presente, può essere espresso in termini di exist

in []: let contains x lis = exists (fun y → x = y) lis

FOR ALL

POSSIAMO DEFINIRE UNA FUNZIONE CHE TESTA UN PREDICATO SU TUTTI GLI ELEMENTI

let tuttiZero lis = forall (x → x = 0) lis ; j

Filter = filtra gli elementi della lista secondo una condizione

- Restituisce gli elementi della lista che lo soddisfanno

in []: let estraiPositivi lis = filter (fun x → x > 0) lis

MAP = data una lista applico una funzione a tutti gli elementi

- I risultati vanno dentro una nuova lista

let rec MAP f lis =

match lis with

| [] → []

| x :: lis' → f x :: MAP f lis'; j

esempio d'uso

```
in[]:map (fun x->x+1) [1;2;3];;
```

```
out[] -> int list = [2;3;4]
```

1) le funzioni viste finora lavorano sui singoli elementi delle liste in modo indipendente
però ovvio che vogliamo elaborare tutti gli elementi es. somma, min, max

Fold-Right

esempio JAVASCRIPT

```
function somma(a) {
```

```
  var s=0
```

```
  for (var i in a)
```

```
    s+=a[i]
```

```
  return s
```

```
}
```

l'importante è capire:

- quali variabili "portarsi dietro" nel ciclo
- come aggiornare col ogni iterazione

• il risultato parziale non sarà memorizzato in una variabile

ma passerà da una chiamata all'altra come parametro o valore di ritorno

```
in[]:let rec somma lis =
```

```
  match lis with
```

```
  | [] -> 0
```

```
  | x::lis' -> x + somma lis';;
```

```
  somma [3;2;4]
```

```
out[]: -> int = 9
```

Fold_right

PARA ciò che ha fatto la somma generalizzandola

```
in []: let rec fold_right f lis a =  
  match lis with  
  | [] -> a  
  | x::lis' -> f x (fold_right f lis' a)
```

Annotations:
- **f**: funzione
- **a**: caso base

esempio

```
in []: let somma lis = fold_right (+) lis 0  
  somma [3;2;4]
```

Annotation:
- **(+)**: operatore che prende 2 valori successivi e fa la somma, vale per tutti gli operatori

```
out []: - : int = 9
```

stringa + lunghezza

```
in []: let stringa_piu_lunga lis =  
  let f x s =  
    if string.length x > string.length s then x else s  
  
  in  
    fold_right f lis "";
```

calcolare min e max di una lista

si prende 1° elemento e si usa come caso base

```
let f x (min, max) =  
  if x < min then (x, max)  
  else if x > max then (min, max)  
  else (min, max)  
  
in  
  match lis with  
  | [] -> (0, 0)  
  | x::lis' -> fold_right f lis' (x, x);;
```

Fold-Left

summa può essere fatta anche così:

in []: let rec summa a lis =

match lis with

[] → a

| x :: lis' → summa (A+x) lis';;

summa 0 [3;2;4]